Instituto Tecnológico de Costa Rica
Escuela de Matemática
Prof.: Juan Pablo Soto Quirós

Semestre II, 2026
CE-1111: Análisis Numérico para Ingeniería
Valor Porcentual: 35 %

Portafolio Computacional

Instrucciones Generales

- La presente asignación corresponde al **Portafolio Computacional** del curso, el cual se desarrollará a lo largo del semestre mediante **siete bloques de entrega**. Cada bloque estará asociado con los contenidos desarrollados y analizados en el curso durante el período correspondiente.
- El Portafolio Computacional corresponde a un **35 % de la nota final del curso**.
- Esta asignación se realiza en grupos de máximo tres personas. Cada grupo debe registrar los nombres de sus integrantes y sus respectivos correos electrónicos en el siguiente enlace

  https://docs.google.com/spreadsheets/d/1Pj1o5k_GsD44Pjb_NJt5DOXiJ03OB43-

- El objetivo del Portafolio Computacional es integrar los conceptos teóricos estudiados en el curso con su **implementación computacional**, permitiendo analizar el comportamiento de los diferentes métodos numéricos mediante experimentos, resultados numéricos, tablas, gráficas y otros recursos computacionales cuando corresponda.
- Para las implementaciones computacionales del Portafolio se utilizará **Python** o **GNU Octave**, según se indique en cada bloque. No se aceptarán soluciones implementadas en R, C++ u otros lenguajes de programación, salvo indicación expresa del profesor.
- Las implementaciones deberán utilizar los **métodos, algoritmos y procedimientos estudiados en clase**. Cuando un ejercicio solicite implementar un método numérico, no se permitirá sustituir dicha implementación por una función de biblioteca que resuelva directamente el problema, salvo que el enunciado lo autorice explícitamente.
- El código desarrollado deberá ser **claro, ordenado y legible**. Se deberán utilizar nombres adecuados para las variables y funciones, así como comentarios cuando estos sean necesarios para facilitar la comprensión de la implementación.
- En los ejercicios que requieran análisis de resultados, no será suficiente presentar únicamente el código o la salida obtenida. El estudiante deberá incluir las **observaciones, comparaciones, interpretaciones o conclusiones** solicitadas en cada ejercicio.
- Cuando se soliciten gráficas, tablas o resultados numéricos, estos deberán presentarse de manera clara y correctamente identificados. Las gráficas deberán incluir, cuando corresponda, título, etiquetas de los ejes, leyenda y demás elementos necesarios para su adecuada interpretación.

- El Portafolio Computacional está organizado en **seis bloques**, cada uno con su respectiva fecha límite de entrega. La hora máxima de recepción para todas las entregas será a las **11:59 p.m.** Las fechas correspondientes se detallan en la siguiente tabla:

Bloque	**Fecha límite de entrega**
1	Domingo 30 de agosto de 2026
2	Jueves 17 de setiembre de 2026
3	Jueves 8 de octubre de 2026
4	Jueves 22 de octubre de 2026
5	Jueves 5 de noviembre de 2026
6	Lunes 30 de noviembre de 2026

Relación con los Atributos de Egreso: Análisis de Problemas (AP)

Esta asignación tiene como propósito fortalecer la capacidad del estudiante para analizar y resolver problemas complejos de ingeniería numérica mediante la implementación computacional de los métodos estudiados en clase. Se vincula directamente con el atributo de egreso **"Análisis de Problemas (AP)"**, relacionado con la identificación, formulación y resolución de problemas de ingeniería mediante principios matemáticos, científicos y de ingeniería.

La relación entre los indicadores de este atributo y la asignación se presenta en la siguiente tabla.

Indicador	**Relación con la Evaluación**
AP1: Identifica un problema complejo de ingeniería utilizando principios de matemáticas, ciencias naturales y ciencias de la ingeniería, integrando aspectos para el desarrollo sostenible.	En cada ejercicio, el estudiante debe identificar el problema matemático o computacional que subyace en la formulación: encontrar raíces de funciones no lineales, resolver sistemas lineales o aproximar soluciones de EDOs, lo cual requiere aplicar principios fundamentales de cálculo, álgebra y análisis numérico.
AP2: Analiza el contexto y las variables relacionadas con el problema complejo de ingeniería identificado, integrando aspectos para el desarrollo sostenible.	El estudiante analiza la naturaleza de cada problema: tipo de método, propiedades de la función o sistema, tolerancia, eficiencia computacional y convergencia. También debe considerar aspectos prácticos como el uso adecuado de recursos computacionales y la calidad de las soluciones.
AP3: Formula un plan de solución para el problema complejo de ingeniería, integrando aspectos para el desarrollo sostenible.	El estudiante diseña, implementa y documenta algoritmos en GNU Octave o Python, seleccionando el método más adecuado, estableciendo condiciones de parada, y validando los resultados mediante gráficas, pruebas de errores y comparación con soluciones exactas cuando existan.

Cuadro 1: Conexión entre el atributo *Análisis de Problemas (AP)* y esta asignación.

Portafolio Computacional – Bloque 1

Instrucciones del Bloque 1

- Todos los ejercicios de programación deberán desarrollarse **en Python**.
- Cuando se solicite implementar explícitamente un método, el estudiante deberá programar el algoritmo correspondiente y no utilizar funciones de Python que calculen directamente la solución.
- Los resultados computacionales deberán presentarse con la precisión y tolerancia indicadas en cada ejercicio.
- Cuando se soliciten comparaciones entre métodos, deberán considerarse los criterios especificados en el enunciado, tales como **número de iteraciones, error o tiempo de ejecución**.
- Las gráficas y tablas solicitadas deberán presentarse de forma clara y debidamente identificada.
- El código utilizado deberá formar parte de la entrega y deberá ser suficientemente claro para permitir su revisión y ejecución.
- Los documentos desarrollados deben estar en una carpeta principal con nombre `Portafolio - Grupo #`, donde `#` es el número de cada grupo.Dicha carpeta debe comprimirse en un archivo `.zip` y subirlo al formulario que se encuentra en el siguiente enlace:

  https://forms.gle/4xkpc8sg5ixo7xoTA

  Observación: Se necesita tener una cuenta de **Gmail** para llenar el formulario.
- **Fecha límite de entrega:** Domingo 30 de agosto de 2026, a las **11:59 p.m.**

Preguntas del Bloque 1

Parte I: Implementación de los métodos

- Implemente computacionalmente los siguientes métodos para la aproximación de soluciones de ecuaciones no lineales: **Newton–Raphson**, **Secante**, **Steffensen**, **Müller**, **Bisección, Falsa Posición.** Todas las funciones deberán almacenarse en un archivo denominado `parte1.py`
- El archivo `parte1.py` deberá contener **únicamente las funciones correspondientes a los seis métodos numéricos**.
- Cada método deberá implementarse mediante una función independiente. Las funciones deberán recibir como entradas, según corresponda al método:
  - la función $f(x)$, ingresada en formato de texto;
  - el valor o los valores iniciales requeridos por el método;
  - el número máximo de iteraciones;
  - la tolerancia.

- Cada función deberá retornar cuatro valores: x_k, er_k, k, $conv$, donde x_k representa la aproximación obtenida, er_k el error correspondiente a la última iteración realizada, k el número de iteraciones ejecutadas y $conv$ una variable que indique si el método alcanzó la tolerancia solicitada antes de llegar al número máximo de iteraciones. Se utilizará la siguiente convención:

$$conv = \begin{cases} 1, & \text{si el método converge antes de alcanzar el máximo de iteraciones,} \\ 0, & \text{en caso contrario.} \end{cases}$$

- Las funciones deberán programar explícitamente los algoritmos estudiados en clase. **No se permite utilizar funciones de Python o bibliotecas externas que resuelvan directamente la ecuación no lineal.**

Parte II: Comparación computacional de los métodos

- Considere la ecuación no lineal

$$\ln(x^2 + 1) - \cos(x) = 4x^2 - 10.$$

- Utilice cada uno de los seis métodos implementados en la Parte I para aproximar **una misma solución** de esta ecuación. Para todos los métodos utilice `iterMax` $= 1000$ y `tol` $= 10^{-8}$
- Cada grupo deberá seleccionar los valores iniciales necesarios para cada método. Se deberá **indicar y justificar explícitamente cómo fueron seleccionados dichos valores iniciales**. La selección deberá sustentarse mediante un análisis previo de la función y no únicamente mediante prueba y error.
- El desarrollo completo de esta parte deberá realizarse en un archivo denominado `parte2.py`
- El archivo `parte2.py` deberá importar y utilizar las funciones desarrolladas en `parte1.py`. **No se deberán implementar nuevamente los métodos numéricos en este archivo.**
- Una vez realizadas las aproximaciones, construya una **tabla comparativa** que incluya, para cada método:
  - la aproximación obtenida x_k;
  - el error final er_k;
  - el número de iteraciones k;
  - el tiempo de ejecución;
  - el indicador de convergencia $conv$.
- La tabla comparativa deberá **mostrarse directamente en la ventana de comandos o terminal como parte de la ejecución del programa**, con un formato claro y legible que permita identificar cada método y sus respectivos resultados.
- Además, genere gráficas que permitan comparar entre los seis métodos:
  - los errores obtenidos;
  - los tiempos de ejecución;
  - el número de iteraciones.

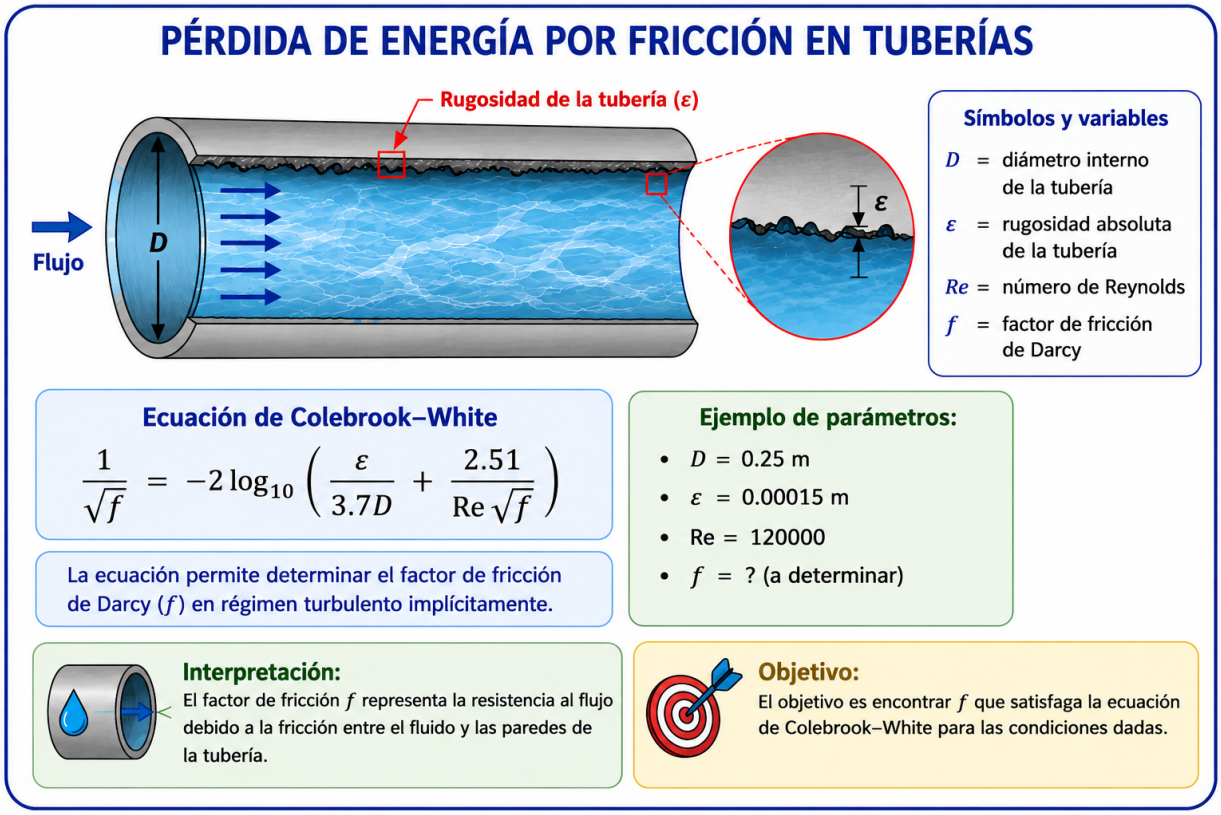
- Las gráficas deberán estar correctamente identificadas y utilizar escalas adecuadas para facilitar la comparación entre los métodos. Cuando las magnitudes presenten diferencias importantes de orden, considere el uso de una **escala logarítmica**.
- Finalmente, realice un **análisis comparativo de los resultados**. No es suficiente indicar cuál método obtuvo el menor tiempo de ejecución o realizó menos iteraciones; deberán discutirse las diferencias observadas y relacionarlas con las características de los métodos estudiados. Este análisis deberá **mostrarse al final de la ejecución del código y aparecer directamente en la ventana de comandos o terminal**.

Parte III: Aplicación – Factor de fricción en una tubería

- En el análisis de sistemas de transporte de fluidos es necesario determinar las pérdidas de energía producidas por la fricción entre el fluido y las paredes de una tubería. Para flujo turbulento, una de las expresiones utilizadas para determinar el **factor de fricción de Darcy** f es la ecuación de Colebrook–White:

$$\frac{1}{\sqrt{f}} = -2\log_{10}\left(\frac{\varepsilon}{3.7D} + \frac{2.51}{Re\sqrt{f}}\right),$$

  donde D representa el diámetro interno de la tubería, ε la rugosidad absoluta de la tubería, Re el número de Reynolds y f el factor de fricción que se desea determinar.
- Considere una tubería con las siguientes características: $D = 0.25$ m, $\varepsilon = 0.00015$ m, $Re = 120000$.



- El desarrollo completo de esta parte deberá realizarse en un archivo denominado `parte3.py`
- El archivo `parte3.py` deberá importar y utilizar las funciones desarrolladas en `parte1.py`. **No se deberán implementar nuevamente los métodos numéricos en este archivo.**

- Realice lo siguiente:

  1. Transforme la ecuación de Colebrook–White en un problema de búsqueda de raíces de la forma $g(f) = 0$.
  2. Analice la función $g(f)$ y determine un intervalo físicamente razonable en el cual buscar el factor de fricción. Recuerde que necesariamente $f > 0$.
  3. Seleccione y justifique los valores iniciales que utilizará para cada uno de los seis métodos. En los métodos que requieren un intervalo inicial, deberá verificarse explícitamente la condición necesaria para iniciar el procedimiento.
  4. Utilizando las funciones desarrolladas en `parte1.py`, aproxime el factor de fricción mediante los métodos de Newton–Raphson, Secante, Steffensen, Müller, Bisección y Falsa Posición.
  5. Utilice para todos los métodos `iterMax` $= 1000$ y `tol` $= 10^{-8}$.
  6. Construya una tabla comparativa que muestre, para cada método, la aproximación obtenida, el error, el número de iteraciones, el tiempo de ejecución y el indicador de convergencia.
  7. Genere gráficas que permitan comparar los **errores, tiempos de ejecución y número de iteraciones** de los seis métodos.
  8. Analice los resultados obtenidos. En particular, discuta si todos los métodos convergen hacia el mismo valor, cuáles requieren una menor cantidad de iteraciones, cuáles presentan un menor tiempo de ejecución y si los resultados son consistentes con las características teóricas estudiadas para cada método.
  9. Interprete el resultado en el contexto del problema. Indique cuál es el **factor de fricción de Darcy aproximado** de la tubería y explique brevemente qué representa esta cantidad dentro del modelo.
  10. Este análisis e interpretación deberá **mostrarse al final de la ejecución del código y aparecer directamente en la ventana de comandos o terminal**.

Portafolio Computacional – Bloque 2

Instrucciones del Bloque 2

- Todos los ejercicios de programación deberán desarrollarse **en GNU Octave**.
- Cuando se solicite implementar explícitamente un método, el estudiante deberá programar el algoritmo correspondiente y no utilizar funciones de GNU Octave que calculen directamente la solución del sistema de ecuaciones.
- Cuando se soliciten comparaciones entre métodos, deberán considerarse principalmente el **error** y el **tiempo de ejecución**. Para los métodos iterativos deberá reportarse adicionalmente el número de iteraciones realizadas.
- Las gráficas y tablas solicitadas deberán presentarse de forma clara y debidamente identificada.
- El código utilizado deberá formar parte de la entrega y deberá ser suficientemente claro para permitir su revisión y ejecución.
- Los documentos desarrollados deben estar en una carpeta principal con nombre `Portafolio - Grupo #`, donde `#` es el número de cada grupo. Dicha carpeta debe comprimirse en un archivo `.zip` y subirlo al formulario que se encuentra en el siguiente enlace:

  https://forms.gle/4xkpc8sg5ixo7xoTA

  Observación: Se necesita tener una cuenta de **Gmail** para llenar el formulario.

Preguntas del Bloque 2

Parte I: Implementación de los métodos

- Implemente computacionalmente los siguientes métodos vistos en clases para la solución de sistemas de ecuaciones lineales: **Eliminación Gaussiana, Factorización LU, Cholesky, QR, Thomas, Jacobi y Gauss–Seidel.** Cada método deberá implementarse **como se explicó en las lecciones del curso** mediante una función independiente.
- Adicionalmente, investigue e implemente el **método iterativo del Gradiente Conjugado** para la solución de sistemas de ecuaciones lineales. Como parte de esta actividad, deberá elaborar un documento en formato PDF denominado `gradiente_conjugado.pdf`, en el cual se incluya:
  - una explicación clara de en qué consiste el método del Gradiente Conjugado;
  - las condiciones que debe cumplir la matriz de coeficientes para que el método sea aplicable;
  - un pseudocódigo del algoritmo;
  - un ejemplo completo de aplicación en un sistema de ecuaciones lineales cuya matriz de coeficientes sea de tamaño 4×4.
- Todas las funciones correspondientes deberán almacenarse en un archivo denominado `parte1.m`. **Nota:** Haga clic [aquí] para consultar una infografía sobre cómo definir varias funciones en un mismo archivo `.m` y utilizarlas desde otros archivos.

- El archivo `parte1.m` deberá contener **únicamente las funciones correspondientes a los métodos numéricos solicitados**.
- Las funciones deberán recibir como entradas los datos necesarios para resolver un sistema de ecuaciones lineales $A\mathbf{x} = \mathbf{b}$.
- En los métodos directos, la función deberá retornar al menos la aproximación $\mathbf{x}$ obtenida.
- En los métodos iterativos de Jacobi y Gauss–Seidel, las funciones deberán recibir además:
  - un vector inicial $\mathbf{x}^{(0)}$;
  - el número máximo de iteraciones `iterMax`;
  - la tolerancia `tol`.

  Estas funciones deberán retornar cuatro valores: $\mathbf{x}_k$, er_k, k y $conv$, donde $\mathbf{x}_k$ representa la aproximación obtenida, $er_k = \|A\mathbf{x}_k - \mathbf{b}\|_2$, k representa el número de iteraciones ejecutadas y $conv$ indica si el método alcanzó la tolerancia solicitada antes de llegar al número máximo de iteraciones. Se utilizará la siguiente convención:
$$conv = \begin{cases} 1, & \text{si } \|A\mathbf{x}^{(k)} - \mathbf{b}\|_2 < \texttt{tol}, \\ 0, & \text{en caso contrario.} \end{cases}$$
- Las funciones deberán programar explícitamente los algoritmos estudiados en clase. **No se permite utilizar funciones de GNU Octave o bibliotecas externas que resuelvan directamente el sistema de ecuaciones o realicen directamente las factorizaciones solicitadas.**
- Se asumirá que las matrices suministradas a cada método cumplen **a priori** las condiciones necesarias para su aplicación. Por tanto, los programas **no deberán incluir verificaciones** de propiedades como estructura tridiagonal, simetría, definición positiva o condiciones de convergencia de los métodos iterativos, con el fin de evitar cálculos adicionales que aumenten el tiempo de ejecución. Será responsabilidad de cada grupo verificar previamente que la matriz utilizada sea adecuada para el método correspondiente.

Parte II: Comparación computacional de los métodos

- Considere el sistema de ecuaciones lineales $A\mathbf{x} = \mathbf{b}$, donde $A \in \mathbb{R}^{300\times 300}$ es la matriz tridiagonal definida por
$$A = \begin{pmatrix} 4 & -1 & & & \\ -1 & 4 & -1 & & \\ & -1 & 4 & \ddots & \\ & & \ddots & \ddots & -1 \\ & & & -1 & 4 \end{pmatrix} \quad \text{y} \quad \mathbf{b} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} \in \mathbb{R}^{300}.$$
  La matriz y el vector deberán generarse directamente mediante GNU Octave y no ingresarse elemento por elemento.
- Resuelva el sistema utilizando los métodos implementados en la Parte I: Eliminación Gaussiana, LU, Cholesky, QR, Thomas, Jacobi, Gauss–Seidel y Gradiente Conjugado.
- Para Jacobi y Gauss–Seidel utilice $\mathbf{x}^{(0)} = \mathbf{0} \in \mathbb{R}^{3000}$ (vector nulo), `iterMax` $= 10000$, y `tol` $= 10^{-8}$.
- El desarrollo completo de esta parte deberá realizarse en un archivo denominado `parte2.m`.

- El archivo `parte2.m` deberá importar y utilizar las funciones desarrolladas en `parte1.m`. **No se deberán implementar nuevamente los métodos numéricos en este archivo.**
- Para cada método, calcule el error mediante $er = \|A\mathbf{x} - \mathbf{b}\|_2$.
- Mida el **tiempo de ejecución** requerido por cada método para obtener la solución del sistema.
- Construya una **tabla comparativa** que incluya, para cada método:
  - el error $\|A\mathbf{x} - \mathbf{b}\|_2$;
  - el tiempo de ejecución.

  Para Jacobi, Gauss–Seidel, y gradiente conjugado deberán mostrarse adicionalmente:
  - el número de iteraciones k;
  - el indicador de convergencia *conv*.
- La tabla comparativa deberá **mostrarse directamente en la ventana de comandos o terminal como parte de la ejecución del programa**, con un formato claro y legible.
- Genere gráficas que permitan comparar entre los métodos:
  - los errores $\|A\mathbf{x} - \mathbf{b}\|_2$;
  - los tiempos de ejecución.

  Cuando las diferencias entre los valores sean considerables, utilice una **escala logarítmica** para facilitar su comparación.
- Para los métodos iterativos, genere adicionalmente una gráfica que permita comparar el número de iteraciones requeridas.
- Finalmente, realice un **análisis comparativo de los resultados**. No es suficiente indicar cuál método obtuvo el menor tiempo o el menor error. Deberán relacionarse los resultados obtenidos con las características de los métodos y con la estructura particular de la matriz A.
- Este análisis deberá **mostrarse al final de la ejecución del código y aparecer directamente en la ventana de comandos o terminal**.

Parte III: Aplicación – Distribución de temperatura en una barra metálica

- Considere una barra metálica delgada de longitud $L = 1$ m. El extremo izquierdo de la barra se mantiene a una temperatura constante de 20°C y el extremo derecho a 50°C.

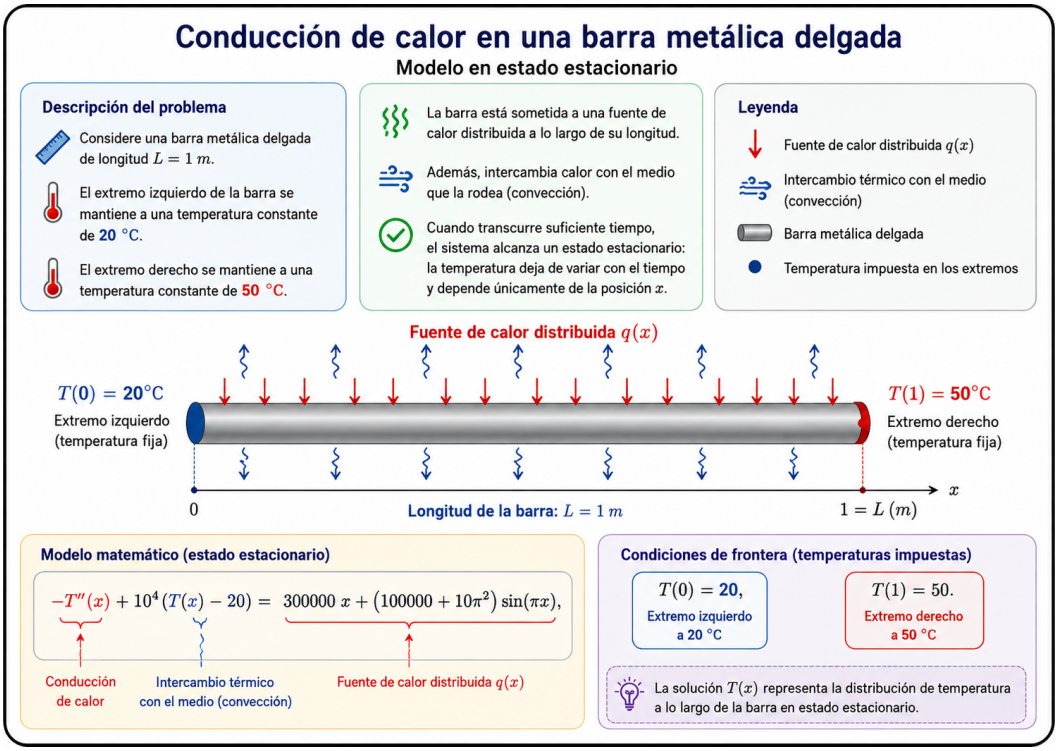
  La barra se encuentra sometida a una fuente de calor distribuida y a un mecanismo de intercambio térmico con el medio que la rodea. Cuando transcurre suficiente tiempo, el sistema alcanza un **estado estacionario**; es decir, la temperatura deja de variar con el tiempo y depende únicamente de la posición a lo largo de la barra.

  Para este problema, la temperatura $T(x)$ se modela mediante el siguiente problema de valores en la frontera:

$$-T''(x) + 10^4\Big(T(x) - 20\Big) = 300000x + \Big(100000 + 10\pi^2\Big)\sin(\pi x),$$

  con $0 < x < 1$, y valores iniciales $T(0) = 20$ $T(1) = 50$.

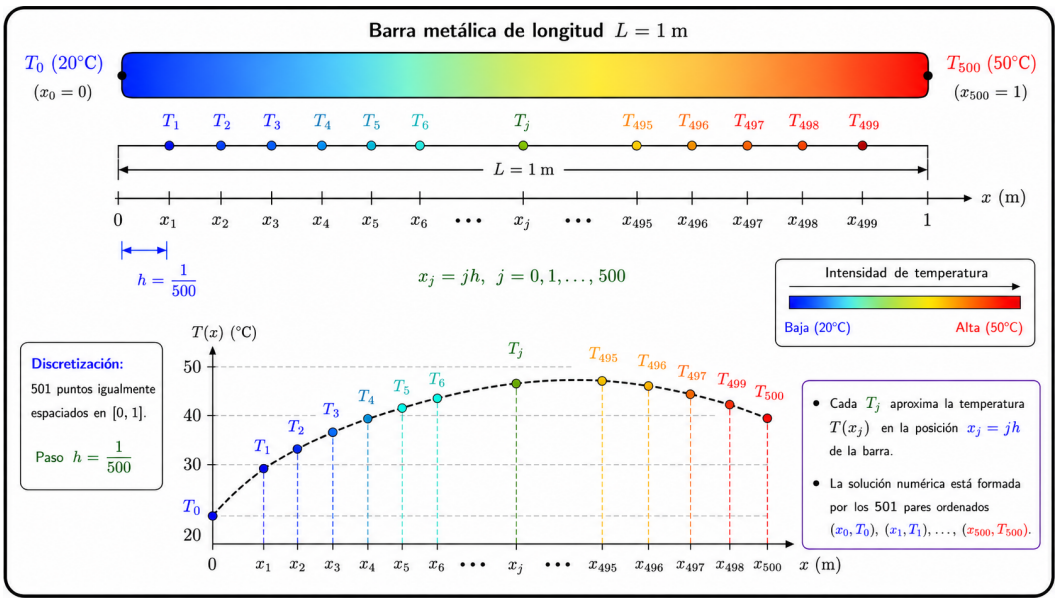
- El propósito de esta parte consiste en aproximar la distribución de temperatura a lo largo de la barra mediante el método de **diferencias finitas** y resolver el sistema de ecuaciones lineales resultante utilizando los métodos estudiados.



- Para aproximar la solución del problema de valores en la frontera, se considerarán 501 puntos igualmente espaciados en el intervalo $[0, 1]$. El tamaño de paso está dado por $h = \frac{1}{500}$, y los puntos se definen como $x_i = ih$, para $i = 0, 1, \ldots, 500$. De esta manera, la solución del problema se aproximará mediante los 501 pares ordenados

$$(x_0, T_0), (x_1, T_1), \ldots, (x_{500}, T_{500}),$$

donde T_i representa una aproximación de la temperatura $T(x_i)$.



- Las condiciones de frontera proporcionan directamente los valores $T_0 = 20$ y $T_{500} = 50$. Por tanto, únicamente es necesario calcular las temperaturas interiores $T_1, T_2, \ldots, T_{499}$.

- Para obtener las temperaturas interiores, deberá resolverse el sistema de ecuaciones lineales

$$A\mathbf{T} = \mathbf{b},$$

donde

$$\mathbf{T} = \begin{pmatrix} T_1 \\ T_2 \\ \vdots \\ T_{499} \end{pmatrix} \in \mathbb{R}^{499},$$

y la matriz $A \in \mathbb{R}^{499\times 499}$ está dada explícitamente por

$$A = \begin{pmatrix} 510000 & -250000 & 0 & \cdots & 0 \\ -250000 & 510000 & -250000 & \ddots & \vdots \\ 0 & -250000 & 510000 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -250000 \\ 0 & \cdots & 0 & -250000 & 510000 \end{pmatrix}.$$

El vector $\mathbf{b} \in \mathbb{R}^{499}$ se construye a partir de

$$\mathbf{b} = \begin{pmatrix} g_1 + 5000000 \\ g_2 \\ g_3 \\ \vdots \\ g_{498} \\ g_{499} + 12500000 \end{pmatrix},$$

donde

$$g_i = 300000x_i + \left(100000 + 10\pi^2\right)\sin(\pi x_i) + 200000,$$

con $x_i = \frac{i}{500}$, para todo $i = 1, \ldots, 499$.

- Resuelva el sistema anterior utilizando los métodos indicados en este bloque. Una vez obtenidos los valores $T_1, T_2, \ldots, T_{499}$, incorpore las condiciones de frontera $T_0 = 20$ y $T_{500} = 50$ para obtener los 501 pares ordenados (x_i, T_i), para $i = 0, 1, \ldots, 500$, que aproximan la distribución de temperatura a lo largo de la barra.

- La solución exacta del problema de valores en la frontera está dada por

$$T(x) = 20 + 30x + 10\sin(\pi x).$$

Esta solución deberá utilizarse **únicamente para corroborar gráficamente** el comportamiento de la aproximación obtenida.

- El desarrollo completo de esta parte deberá realizarse en un archivo denominado `parte3.m`.

- El archivo `parte3.m` deberá importar y utilizar las funciones desarrolladas en `parte1.m`. **No se deberán implementar nuevamente los métodos numéricos estudiados en clase en este archivo.**
- Utilice los métodos de Eliminación Gaussiana, LU, Cholesky, QR, Thomas, Jacobi, Gauss–Seidel y gradiente conjugado para resolver el sistema obtenido.
- Para Jacobi, Gauss–Seidel y gradiente conjugado utilice $\mathbf{T}^{(0)} = \mathbf{0} \in \mathbb{R}^{499}$ (vector nulo), `iterMax` $= 10000$ y `tol` $= 10^{-8}$.
- Para cada método calcule el error $er = \|A\mathbf{T} - \mathbf{b}\|_2$ y mida el correspondiente tiempo de ejecución.
- Construya una tabla comparativa que muestre, para cada método:
  - el error $\|A\mathbf{T} - \mathbf{b}\|_2$;
  - el tiempo de ejecución.

  Para los métodos iterativos deberán mostrarse adicionalmente el número de iteraciones y el indicador de convergencia.
- Genere gráficas que permitan comparar:
  - los errores obtenidos por los diferentes métodos;
  - los tiempos de ejecución;
  - el número de iteraciones de los métodos iterativos.

  Cuando sea necesario, utilice una escala logarítmica.
- Genere además una gráfica de la distribución de temperatura en la barra. En una misma figura deberán mostrarse:
  - la solución exacta $T(x) = 20 + 30x + 10\sin(\pi x)$;
  - los pares ordenados (x_i, T_i) correspondientes a una de las soluciones numéricas obtenidas.

  La solución exacta deberá mostrarse mediante una curva continua y la solución numérica deberá representarse de manera que puedan identificarse los puntos producidos por la discretización, usando el comando `scatter`.
- Analice los resultados obtenidos. En particular, discuta:
  - si los diferentes métodos producen soluciones equivalentes desde el punto de vista del residuo $\|A\mathbf{T} - \mathbf{b}\|_2$;
  - cuáles métodos presentan los menores tiempos de ejecución;
  - qué influencia tiene la estructura tridiagonal de la matriz sobre el comportamiento del método de Thomas;
  - qué ventajas presenta Cholesky cuando la matriz satisface las condiciones necesarias para su aplicación;
  - las diferencias observadas entre Jacobi y Gauss–Seidel;
  - si los pares ordenados obtenidos mediante diferencias finitas siguen adecuadamente la solución exacta del problema.
- Este análisis e interpretación deberá **mostrarse al final de la ejecución del código y aparecer directamente en la ventana de comandos o terminal**.

Portafolio Computacional – Bloque 3

Instrucciones del Bloque 3

- Todos los ejercicios de programación deberán desarrollarse **en Python**.
- Cuando se solicite implementar explícitamente un método, el estudiante deberá programar el algoritmo correspondiente y no utilizar funciones de Python que calculen directamente los valores y vectores propios de una matriz.
- En este bloque se trabajará únicamente con matrices **simétricas semidefinidas positivas**. Se asumirá que las matrices suministradas cumplen *a priori* estas condiciones. Por tanto, los programas **no deberán verificar** si una matriz es simétrica o semidefinida positiva, con el fin de evitar cálculos adicionales que aumenten el tiempo de ejecución.
- Bajo las condiciones anteriores, se asumirá que todos los valores propios de las matrices consideradas son **reales y no negativos**.
- Cuando se soliciten comparaciones entre métodos, deberán considerarse principalmente el **error**, el **número de iteraciones** y el **tiempo de ejecución**.
- Las gráficas y tablas solicitadas deberán presentarse de forma clara y debidamente identificada.
- El código utilizado deberá formar parte de la entrega y deberá ser suficientemente claro para permitir su revisión y ejecución.
- Los documentos desarrollados deben estar en una carpeta principal con nombre `Portafolio - Grupo #`, donde `#` es el número de cada grupo. Dicha carpeta debe comprimirse en un archivo `.zip` y subirlo al formulario que se encuentra en el siguiente enlace:

  https://forms.gle/4xkpc8sg5ixo7xoTA

  Observación: Se necesita tener una cuenta de **Gmail** para llenar el formulario.

Preguntas del Bloque 3

Parte I: Implementación de los métodos

- Implemente computacionalmente el **método QR** para la aproximación de los valores y vectores propios de una matriz simétrica semidefinida positiva.
- Adicionalmente, investigue e implemente el **método de Jacobi para valores propios**. Como parte de esta actividad, deberá elaborar un documento en formato PDF denominado `jacobi.pdf`, en el cual se incluya:
  - una explicación clara de en qué consiste el método de Jacobi para aproximar valores y vectores propios;
  - las condiciones que debe cumplir la matriz para la aplicación del método;
  - una explicación de cómo se obtienen las aproximaciones de los valores propios y sus correspondientes vectores propios;

  - un pseudocódigo del algoritmo;
  - un ejemplo completo de aplicación a una matriz simétrica semidefinida positiva de tamaño 4×4.

- Los métodos QR y Jacobi deberán implementarse mediante funciones independientes y almacenarse en un archivo denominado `parte1.py`.
- El archivo `parte1.py` deberá contener **únicamente las funciones correspondientes a los métodos numéricos solicitados**.
- Cada función deberá recibir como entradas:
  - la matriz A;
  - el número máximo de iteraciones `iterMax`;
  - la tolerancia `tol`.
- En ambos métodos, en cada iteración las aproximaciones de los valores propios deberán ordenarse de mayor a menor. Si

$$\boldsymbol{\lambda}^{(k)} = \begin{pmatrix} \lambda_1^{(k)} \\ \lambda_2^{(k)} \\ \vdots \\ \lambda_n^{(k)} \end{pmatrix}, \qquad \lambda_1^{(k)} \geq \lambda_2^{(k)} \geq \cdots \geq \lambda_n^{(k)},$$

  representa el vector formado por las aproximaciones de los valores propios en la iteración k, se utilizará como error

$$er_k = \left\| \boldsymbol{\lambda}^{(k)} - \boldsymbol{\lambda}^{(k-1)} \right\|_2.$$

- Los vectores propios deberán almacenarse como las columnas de la matriz

$$V = \begin{pmatrix} | & | & & | \\ \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_n \\ | & | & & | \end{pmatrix},$$

  de manera que $\mathbf{v}_1$ sea el vector propio correspondiente a λ_1, $\mathbf{v}_2$ corresponda a λ_2, y así sucesivamente.
- Todos los vectores propios deberán estar **normalizados**; es decir,

$$\|\mathbf{v}_j\|_2 = 1, \qquad j = 1, \ldots, n.$$

- Cada función deberá retornar cinco elementos: $\boldsymbol{\lambda}^{(k)}$, $V^{(k)}$, er_k, k, *conv*, donde:
  - $\boldsymbol{\lambda}^{(k)}$ contiene las aproximaciones de los valores propios;
  - $V^{(k)}$ contiene los correspondientes vectores propios;
  - er_k representa el error correspondiente a la última iteración realizada;
  - k representa el número de iteraciones ejecutadas;
  - *conv* indica si el método alcanzó la tolerancia solicitada antes de llegar al número máximo de iteraciones.

Se utilizará la siguiente convención:

$$conv = \begin{cases} 1, & \text{si } er_k < \texttt{tol}, \\ 0, & \text{en caso contrario.} \end{cases}$$

- Se asumirá que las matrices suministradas a los métodos son simétricas semidefinidas positivas. Los algoritmos **no deberán incluir verificaciones** de estas propiedades.
- Las funciones deberán programar explícitamente los algoritmos solicitados. **No se permite utilizar** funciones como `numpy.linalg.eig`, `numpy.linalg.eigh`, `numpy.linalg.eigvals` o cualquier otra función que calcule directamente los valores o vectores propios.
- Para la implementación del método QR se permite utilizar una función de Python para calcular la **factorización QR** de la matriz en cada iteración. Sin embargo, el proceso iterativo para aproximar los valores y vectores propios deberá ser implementado explícitamente.

Parte II: Comparación computacional de los métodos

- Considere la matriz tridiagonal, simétrica y definida positiva

$$A = \begin{pmatrix} 4 & -1 & & & & & & & & \\ -1 & 4 & -1 & & & & & & & \\ & -1 & 4 & -1 & & & & & & \\ & & -1 & 4 & -1 & & & & & \\ & & & -1 & 4 & -1 & & & & \\ & & & & \ddots & \ddots & \ddots & & & \\ & & & & & \ddots & \ddots & \ddots & & \\ & & & & & & -1 & 4 & -1 & \\ & & & & & & & -1 & 4 & -1 \\ & & & & & & & & -1 & 4 \end{pmatrix} \in \mathbb{R}^{200\times 200}.$$

- Aproxime los valores y vectores propios de A utilizando los métodos **QR y Jacobi** implementados en la Parte I.
- Para ambos métodos utilice $\texttt{iterMax} = 10000$ y $\texttt{tol} = 10^{-8}$.
- El desarrollo completo de esta parte deberá realizarse en un archivo denominado `parte2.py`.
- El archivo `parte2.py` deberá importar y utilizar las funciones desarrolladas en `parte1.py`. **No se deberán implementar nuevamente los métodos numéricos en este archivo.**
- Para cada método, muestre los valores propios aproximados ordenados de mayor a menor y la correspondiente matriz de vectores propios normalizados.
- Compruebe numéricamente cada par propio $(\lambda_j, \mathbf{v}_j)$ mediante el residuo

$$r_j = \|A\mathbf{v}_j - \lambda_j \mathbf{v}_j\|_2, \qquad j = 1, \ldots, 200.$$

Reporte además el mayor residuo obtenido:

$$r_{\text{máx}} = \max_{1\le j\le 200} \|A\mathbf{v}_j - \lambda_j \mathbf{v}_j\|_2.$$

- Verifique numéricamente la ortogonalidad de los vectores propios mediante

$$er_{\text{ort}} = \|V^T V - I_{200}\|_F, .$$

  donde I_{200} es la matriz identidad de tamaño 200×200 y $\|\cdot\|_F$ es la norma de Frobenius.

- Mida el **tiempo de ejecución** requerido por cada método.

- Construya una **tabla comparativa** que incluya, para cada método:

  - el error final er_k;
  - el número de iteraciones k;
  - el tiempo de ejecución;
  - el indicador de convergencia *conv*;
  - el residuo máximo $r_{\text{máx}}$;
  - el error de ortogonalidad $\|V^T V - I\|_F$.

- La tabla comparativa deberá **mostrarse directamente en la ventana de comandos o terminal como parte de la ejecución del programa**, con un formato claro y legible.

- Genere gráficas que permitan comparar entre los métodos:

  - el error final er_k;
  - el tiempo de ejecución;
  - el número de iteraciones.

  Cuando las diferencias entre los valores sean considerables, utilice una **escala logarítmica** para facilitar su comparación.

- Finalmente, realice un **análisis comparativo de los resultados**. En particular, discuta:

  - si ambos métodos aproximan los mismos valores propios;
  - el comportamiento de los residuos $\|A\mathbf{v}_j - \lambda_j \mathbf{v}_j\|_2$;
  - la normalización y ortogonalidad de los vectores propios obtenidos;
  - las diferencias observadas en el número de iteraciones y el tiempo de ejecución;
  - las diferencias observadas en el criterio de parada de ambos métodos.

- Al comparar vectores propios obtenidos mediante métodos diferentes, recuerde que el signo de un vector propio puede cambiar sin modificar su validez. Por tanto, dos vectores $\mathbf{v}$ y $-\mathbf{v}$ representan el mismo vector propio para efectos de este problema.

- Este análisis deberá **mostrarse al final de la ejecución del código y aparecer directamente en la ventana de comandos o terminal**.

Parte III: Aplicación – Reconocimiento facial mediante valores propios

- En esta parte se estudiará una aplicación de los valores y vectores propios al **reconocimiento facial** utilizando la base de datos **AT&T Database of Faces**.
- Antes de realizar esta parte, cada grupo deberá leer cuidadosamente el documento `rec_facial.pdf` suministrado junto con esta asignación.
- El procedimiento descrito en dicho documento deberá utilizarse como guía para construir el modelo de reconocimiento facial. En particular, deberán realizarse los **Pasos 1 al 8** indicados en el documento, respetando la separación establecida entre las fotografías de entrenamiento y las fotografías de prueba.
- Para realizar el cálculo de los valores y vectores propios requerido por el procedimiento, deberá utilizarse **uno de los dos métodos implementados en la Parte I: QR o Jacobi**. El grupo deberá indicar cuál de los dos métodos utilizó, basado en los resultados obtenidos en la Parte II. **Justifique la respuesta**.
- Para ambos métodos utilice `iterMax` $= 10^6$ y `tol` $= 10^{-5}$.
- No se permite utilizar funciones de Python que calculen directamente los valores o vectores propios para sustituir el método seleccionado.
- El desarrollo completo de esta parte deberá realizarse en un archivo denominado `parte3.py`.
- El archivo `parte3.py` deberá importar y utilizar las funciones desarrolladas en `parte1.py`. **No se deberá implementar nuevamente el método de valores propios seleccionado en este archivo.**
- Después de realizar los Pasos 1 al 8 del documento, complete los siguientes pasos.

I. Evaluar las 40 fotografías de prueba

Aplique el procedimiento de identificación a los archivos

`s1/10.png`, `s2/10.png`, ..., `s40/10.png`.

Registre los resultados obtenidos en una tabla con la siguiente estructura:

Fotografía	Persona real	Persona predicha	¿Correcta?
`s1/10.png`	1	...	...
`s2/10.png`	2	...	...
⋮	⋮	⋮	⋮
`s40/10.png`	40	...	...

Calcule la exactitud del procedimiento mediante

$$\text{Exactitud} = 100\frac{\text{número de identificaciones correctas}}{40}.$$

El número k de caras base seleccionado utilizando únicamente las fotografías de entrenamiento deberá mantenerse fijo durante toda la etapa de prueba. **No se permite modificar k después de observar los resultados de las fotografías de prueba.**

II. Presentar los resultados

Como parte de la ejecución del programa y del informe correspondiente, deberá presentarse:

1. justificar el método numérico escogido.
2. el rostro promedio y las primeras seis caras base disponibles;
3. el número de valores propios retenidos r, el número de caras base seleccionado k y la proporción de variación conservada;
4. la tabla completa con las predicciones correspondientes a las 40 fotografías de prueba;
5. el número total de identificaciones correctas y la exactitud obtenida;
6. fotografías de prueba junto con la fotografía de entrenamiento más cercana encontrada por el procedimiento de todas las 40 personas;
7. una explicación breve de las confusiones o identificaciones incorrectas observadas.

- Recuerde que todas las personas utilizadas en la etapa de prueba aparecen también en el conjunto de entrenamiento mediante otras fotografías. Por tanto, esta aplicación evalúa la identificación entre las **40 personas conocidas** y no considera el problema de detectar o rechazar personas que no pertenezcan a la base de datos.
- Finalmente, realice un **análisis de los resultados obtenidos**, relacionando el comportamiento del procedimiento de reconocimiento facial con el papel que desempeñan los valores y vectores propios en la reducción de la dimensión de las fotografías.
- Este análisis deberá **presentarse en un documento pdf con nombre** `resultados.pdf`.

